ДИСЦИПЛИНА	Основы интеллектуального анализа данных
ИНСТИТУТ	Институт технологий управления
КАФЕДРА	Статистики и математических методов в управлении
ГРУППЫ	ГСБО-01-21, ГСБО-01-22,
ВИДЫ УЧЕБНОГО МАТЕРИАЛА	П5. Тема – Решение задачи регрессии средствами языка R
ПРЕПОДАВАТЕЛЬ	Бурцева Т.А.
СЕМЕСТР	5 семестр

Методические указания

Выполнение работы предполагает реализацию примера на данных файла reg-18.xls, чтобы понять логику методов и получить практические навыки их применения в R-Studio, а также разработку собственной модели. В качестве исходных данных для разработки собственной модели предлагаются данные из файла reg-18.xls, предпочтительней использовать данные, которые студент выбрал сам. Выбор независимых переменных для самостоятельной модели из файла reg-18.xls реализуется случайным образом (set=sample(1:18, round(0.2*18))), зависимая переменная У одна для всех. Обязательна реализация всех рассмотренных в примере моделей. Переменные для моделирования следует очистить.

Пример выполнения работы

Шаг 0. Загружаем нужные пакеты

```
#пакет создания и использования моделей всех типов
install.packages("tidyverse")
#пакет инструментов построения статистических моделей в R
install.packages("caret")
#пакет построения графиков в R
install.packages("ggplot2")
#пакет построения сплайновой регрессии
install.packages("splines")
#пакет построения обобщенных аддитивных функции моделирования
#gam , predict.gam и plot.gam
install.packages("mgcv")
# пакет для метода главных компонент
install.packages("knitr")
#пакет для работы с нейросетью
install.packages("neuralnet")
library(neuralnet)
library(knitr)
library(tidyverse)
library(caret)
library(ggplot2)
library(splines)
library(mgcv)
```

Шаг. 1. Загружаем данные из файла reg-18.xls, который содержит данные об оценке региональной производительности труда (ВРП на 1000 занятых в экономике региона) и ее факторах.

Шаг. 2 С помощью функции lm и пошаговой регрессии строим модель множественной линейной регрессии на системе факторов x5, x7, x11 (этот набор не использовать для самостоятельного решения), используя код:

```
tfull=reg_18[,c(-1,- 2,- 3,- 4, -5, -7, -9, -10, -11, -13, -14, -15, -16, -17, -18, -19)]
fit_full <- lm(tfull$y ~ ., data = tfull)
summary(fit_full)
```

Результат множественной регрессии:

```
lm(formula = tfull$y ~ ., data = tfull)

Residuals:
    Min      1Q  Median      3Q     Max
-743.02 -266.82  -90.96   69.83 2810.31

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.224e+03  6.011e+02  -2.037   0.0452 *
x5           3.976e+01  8.198e+00   4.850 6.54e-06 ***
x7           6.557e+00  1.372e+01   0.478   0.6340
x11          3.570e-05  7.134e-04   0.050   0.9602
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 552.7 on 75 degrees of freedom
Multiple R-squared:  0.2754,   Adjusted R-squared:  0.2464
F-statistic:   9.5 on 3 and 75 DF,  p-value: 2.15e-05
```

```
predictions<- fit_full%>% predict(tfull)
data.frame(
  RMSE=RMSE(predictions,tfull$y),
  R2=R2(predictions,tfull$y)
)
```

Результат:

```
RMSE        R2
1 538.5112 0.2753633
```

Выводы: надежность модели 24,6%, статистически значимый фактор x5. Корень из среднеквадратичной ошибки RSME =1 538.5112 – средняя ошибка модели.

Применим пошаговую регрессию:

```
optimal_fit <-  step(fit_full, direction = "backward")
summary(optimal_fit)
```

Результат:

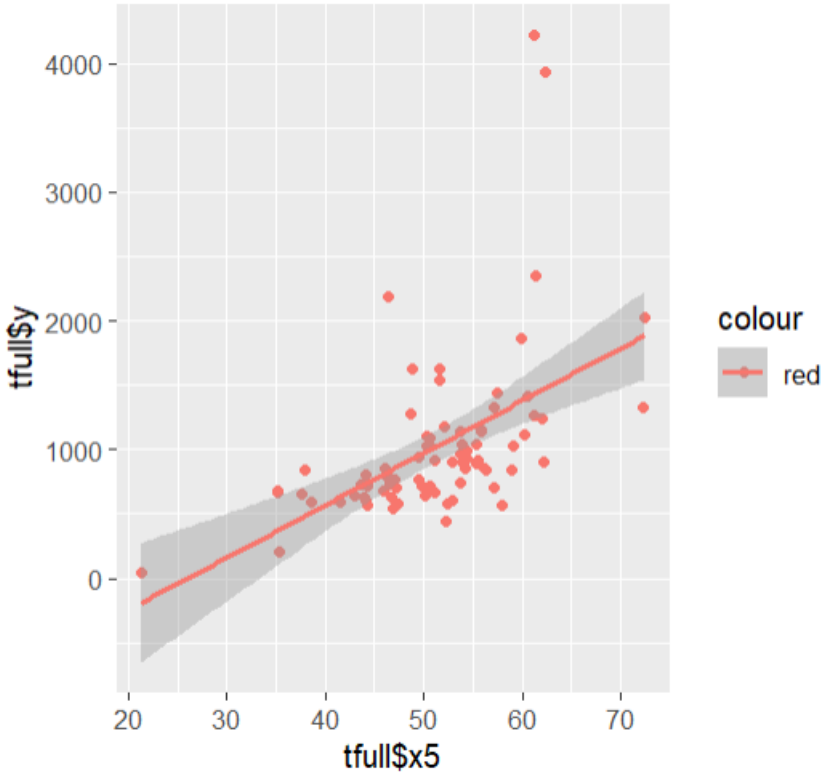
```
Call:
lm(formula = tfull$y ~ x5, data = tfull)
Residuals:
   Min     1Q Median     3Q    Max
-725.98 -276.35  -94.66   75.12 2789.15
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -1055.249    390.629  -2.701  0.00849 **
x5             40.590      7.559   5.370 8.09e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
Residual standard error: 546.6 on 77 degrees of freedom
Multiple R-squared:  0.2724, Adjusted R-squared:  0.263
F-statistic: 28.83 on 1 and 77 DF,  p-value: 8.091e-07
```

```
predictions<- fit_full%>% predict(tfull)
data.frame(
  RMSE=RMSE(predictions,tfull$y),
  R2=R2(predictions,tfull$y)
)
RMSE         R2
1 538.5112 0.2753633
```

Вывод: модель также подобрала парную регрессию с x5.

Визуализируем ее

```
ggplot(tfull, aes(x = tfull$x5, y = tfull$y, col = "red")) +   geom_point()  +  geom_smooth(method = "lm")
```



Вывод: глядя на взаимосвязь, видно, что использование линейной регрессии для этого набора данных не будет правильным выбором. Для этого существуют альтернативные методы, производные от модели линейной регрессии, которые можно использовать для нелинейных взаимосвязей.

Шаг 3. Применим метод главных компонент, который строит регрессию на линейных комбинациях исходных факторов.

```
#Загружаем нужный пакет:
install.packages("knitr")
library("knitr")
#нормируем переменные x, то есть проводим их стандартизацию параметр (scale = TRUE)
#и применяем метод главных компонент для переменных x
hept_pca=prcomp(tfull[,-4], scale = TRUE)
#сохраним в pc главные компоненты
pc <- hept_pca$x
#озаглавим PC
head(pc)
# сохраним веса исходных переменных в главных компонентах в loadings
loadings <- hept_pca$rotation
```

head(loadings)

Ответ:

```
head(loadings)
           PC1         PC2         PC3
x5  -0.5019918  0.78300379 -0.3673000
x7  -0.5648496 -0.61842012 -0.5463529
x11 -0.6549421 -0.06679534  0.7527212
```

То есть первая компонента это $-0{,}5x_5-0{,}56x_7-0{,}65x_{11}$ и т.д.

summary(hept_pca)

```
summary(hept_pca)
Importance of components:
                          PC1    PC2   PC3
Standard deviation     1.2871 0.9161 0.710
Proportion of Variance 0.5522 0.2797 0.168
Cumulative Proportion  0.5522 0.8320 1.000
```

#построим регрессию на первых двух компонентах, так как они описывают более 80% вариации независимых переменных

fit_full <- lm(tfull$y ~ pc[,1]+pc[,2])

summary(fit_full)

Ответ:

```
fit_full <- lm(tfull$y ~ pc[,1]+pc[,2])
> summary(fit_full)

Call:
lm(formula = tfull$y ~ pc[, 1] + pc[, 2])

Residuals:
    Min      1Q  Median      3Q     Max
-633.43 -241.17 -134.54   65.15 2864.48

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1016.13      62.73  16.199  < 2e-16 ***
pc[, 1]      -184.63      49.05  -3.764 0.000327 ***
pc[, 2]       234.01      68.91   3.396 0.001091 **
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 557.6 on 76 degrees of freedom
Multiple R-squared:  0.2527,   Adjusted R-squared:  0.233
F-statistic: 12.85 on 2 and 76 DF,  p-value: 1.559e-05
```

Несмотря на то, что наши переменные имеют отличные от нуля коэффициенты в уравнении модели, надежность ее по-прежнему низкая, 23,3%.

Шаг 4. Строим модели множественной нелинейной регрессии на системе факторов x_5, x_7, x_{11}.

4.1. Полиномиальная регрессия получается путем взятия степеней независимой переменной. Здесь важно то, в какой степени это нужно принимать. Например, модель линейной регрессии для генерации полиномиальной регрессии 2-й степени показана ниже.

$$y = b_0 + b_1 * x_5 + b_2 * x_5^2$$

Смоделируем наш набор данных с помощью полиномиальной регрессии 2-го порядка и посмотрим на итоговый результат:

model<-lm(tfull$y~poly(tfull$x5,2), data=tfull)

summary(model)

результат:

```
Call:
lm(formula = tfull$y ~ poly(tfull$x5, 2), data = tfull)

Residuals:
    Min      1Q  Median      3Q     Max
-804.97 -255.52  -77.87   71.55 2749.89
```

```
Coefficients:
                    Estimate Std. Error t value Pr(>|t|)
(Intercept)         1016.13      61.38  16.554  < 2e-16 ***
poly(tfull$x5, 2)1  2934.83     545.57   5.379 7.98e-07 ***
poly(tfull$x5, 2)2   617.25     545.57   1.131    0.261
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 545.6 on 76 degrees of freedom
Multiple R-squared:  0.2845,   Adjusted R-squared:  0.2657
F-statistic: 15.11 on 2 and 76 DF,  p-value: 2.988e-06
```

```
predictions<- model%>% predict(tfull)
data.frame(
  RMSE=RMSE(predictions,tfull$y),
  R2=R2(predictions,tfull$y)
)
RMSE        R2
1 535.1092 0.2844901
```

Вывод качество улучшилось, но все равно оно недостаточное.

4.2. Связь между переменными можно смоделировать, применив логарифмическое преобразование к независимой переменной:

```
fit_full <- lm(tfull$y ~ log(tfull$x5), data = tfull)
summary(fit_full)
predictions<- fit_full%>% predict(tfull)
data.frame(
  RMSE=RMSE(predictions,tfull$y),
  R2=R2(predictions,tfull$y)
)
```

Ответ:

```
Call:
lm(formula = tfull$y ~ log(tfull$x5), data = tfull)

Residuals:
    Min      1Q  Median      3Q     Max
-695.78 -272.24 -130.23   68.08 2853.98

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    -5935.6     1394.6  -4.256 5.80e-05 ***
log(tfull$x5)   1774.3      355.6   4.990 3.66e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 557 on 77 degrees of freedom
Multiple R-squared:  0.2443,   Adjusted R-squared:  0.2345
F-statistic:  24.9 on 1 and 77 DF,  p-value: 3.657e-06
```

```
> predictions<- fit_full%>% predict(tfull)
> data.frame(
+   RMSE=RMSE(predictions,tfull$y),
+   R2=R2(predictions,tfull$y)
+ )
      RMSE       R2
1 549.9152 0.244347
```

Вывод качество не улучшилось, но все равно оно недостаточное.

4.3. Сплайн-регрессия, когда модель создается путем получения степеней. При сплайн-регрессии обычно устанавливается модель с 3-й степенью.

```
knots<-quantile(tfull$x5 ,p=c(0.10,0.50,0.90))
model<-lm(tfull$y~bs(tfull$x5, knots=knots),data=tfull)
summary(model)
```

ответ:

```
lm(formula = tfull$y ~ bs(tfull$x5, knots = knots), data = tfull)

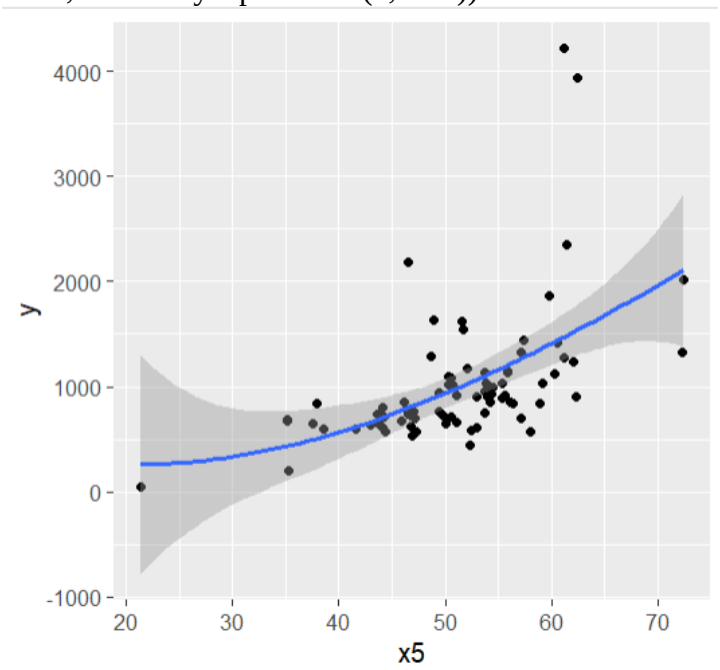
Residuals:
     Min       1Q   Median       3Q      Max
-1492.19  -238.81   -21.94   112.16  2280.11

Coefficients:
                                Estimate Std. Error t value Pr(>|t|)
(Intercept)                        45.15     497.73   0.091   0.9280
bs(tfull$x5, knots = knots)1      953.69    1147.83   0.831   0.4088
bs(tfull$x5, knots = knots)2      100.55     735.42   0.137   0.8916
bs(tfull$x5, knots = knots)3     1231.40     578.43   2.129   0.0367 *
bs(tfull$x5, knots = knots)4      101.96     745.92   0.137   0.8917
bs(tfull$x5, knots = knots)5     6768.10    1440.73   4.698 1.23e-05 ***
bs(tfull$x5, knots = knots)6     1571.01     612.44   2.565   0.0124 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 497.9 on 72 degrees of freedom
Multiple R-squared:  0.4355,   Adjusted R-squared:  0.3885
F-statistic: 9.258 on 6 and 72 DF,  p-value: 1.641e-07
```

Качество улучшилось, визуализируем результат:

```
ggplot(tfull, aes(x5,y))+
  geom_point()+
  stat_smooth(method=lm,formula=y~splines::bs(x,df=3))
```



Вывод: Можно сказать, что эта модель дает лучшие результаты, по сравнению с другими рассмотренными выше методами. В полиномиальной регрессии определяются степени, а в сплайн-регрессии определяются узлы и создаются модели.

4.4. Обобщенные аддитивные модели, основаны на применении функции «гамма» gam. Код в R показан ниже:

```
model<-gam(y~s(x5),data=tfull)
predictions<-model %>% predict(tfull)
data.frame(
  RMSE=RMSE(predictions,tfull$y),
  R2=R2(predictions,tfull$y)
)
summary(model)
```

Ответ:

```
RMSE        R2
1 481.7183 0.4217214
> summary(model)

Family: gaussian
Link function: identity

Formula:
y ~ s(x5)

Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  1016.13      56.93   17.85   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

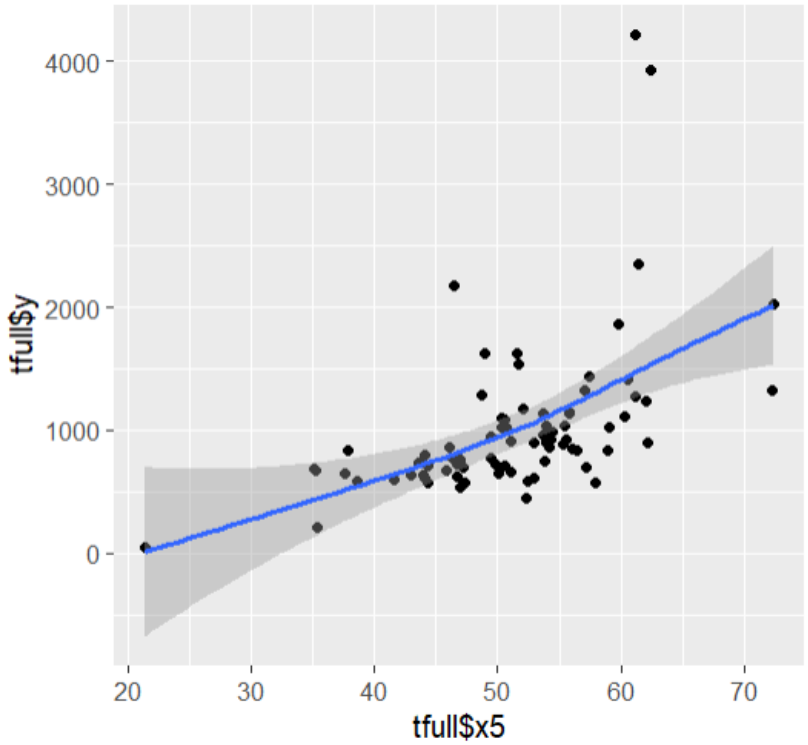
Approximate significance of smooth terms:
        edf Ref.df     F  p-value
s(x5) 6.408  7.414 6.521 4.56e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

R-sq.(adj) =  0.368   Deviance explained =   42%
GCV = 2.8256e+05  Scale est. = 2.5607e+05  n = 79
```

Визуализируем:

```
ggplot(tfull, aes(tfull$x5,tfull$y))+
  geom_point()+
  stat_smooth(method=gam,formula=y~s(x))
```

Ответ:



Вывод: обсудили методы, которые можно использовать для моделирования линейной и нелинейной регрессии, и сравнили их с точки зрения критериев RMSE и R-квадрат. В результате можем сказать, что для этого набора данных предпочтительна обобщенная аддитивная модель регрессии y от x_5.

Шаг 5. Применим нейросеть, так как она также используется для построения регрессионных моделей и не требует выполнения предпосылок теоремы Гаусса-Маркова. Сохраним в data все данные:

```
data=tfull[,1:4]
```

Должны убедиться, что ни одна точка данных не отсутствует, в противном случае нужно исправить набор данных:

```
apply(data,2,function(x) sum(is.na(x)))
```

Ответ:

```
apply(data,2,function(x) sum(is.na(x)))
 x5  x7 x11   y
  0   0   0   0
```

Вывод: пропусков нет.

Случайным образом разбиваем данные на обучающий (train) и тестовый (test) наборы данных. Функция sample (x, size) выводит вектор заданного размера из случайно выбранных выборок из вектора x. По умолчанию выборка производится без замены: index является случайным вектором значений.

```
index <- sample(1:nrow(data),round(0.75*nrow(data)))

train <- data[index,] – обучающая выборка
test <- data[-index,]- тестовая выборка
```

Строим обобщенную модель на обучающей выборке:

```
lm.fit <- glm(train$y~., data=train)
summary(lm.fit)
```

Ответ:

```
glm(formula = train$y ~ ., data = train)

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.589e+03  8.305e+02  -1.913   0.0609 .
x5           4.496e+01  1.071e+01   4.199 9.88e-05 ***
x7           1.091e+01  1.830e+01   0.596   0.5536
x11          1.932e-04  4.378e-03   0.044   0.9650
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for gaussian family taken to be 374123.5)

    Null deviance: 28214057  on 58  degrees of freedom
Residual deviance: 20576794  on 55  degrees of freedom
AIC: 930.4

Number of Fisher Scoring iterations: 2
```

Получаем теоретические значения по модели:

pr.lm <- predict(lm.fit,test)

Для оценки качества модели будем использовать среднеквадратичную ошибку (MSE) как меру того, насколько наши прогнозы далеки от реальных данных.

MSE.lm <- sum((pr.lm - test$y)^2)/nrow(test)

Ответ:

MSE.lm	double [1]	138757.2

#нормируем данные и масштабируем на интервале от 0 до 1:

```
maxs <- apply(data, 2, max)
mins <- apply(data, 2, min)
scaled <- as.data.frame(scale(data, center = mins, scale = maxs - mins))
train_ <- scaled[index,]
test_ <- scaled[-index,]
```

Устанавливаем пакет для работы с нейросетью.

install.packages("neuralnet")

library(neuralnet)

Не существует фиксированного правила относительно количества слоев и нейронов, хотя есть несколько более или менее принятых практических правил. Количество нейронов, должно быть между размером входного слоя (число переменных x) и размером выходного слоя, обычно 2/3 от входного размера. В примере, 3 скрытых слоя. Входной слой имеет 3 входа, а выходной слой, конечно, имеет один выход, так как делаем регрессию.

В n записываем обозначения переменных:

n <- names(train)

В f формулу модели:

f <- as.formula(paste("y ~", paste(n[!n %in% "y"], collapse = " + ")))

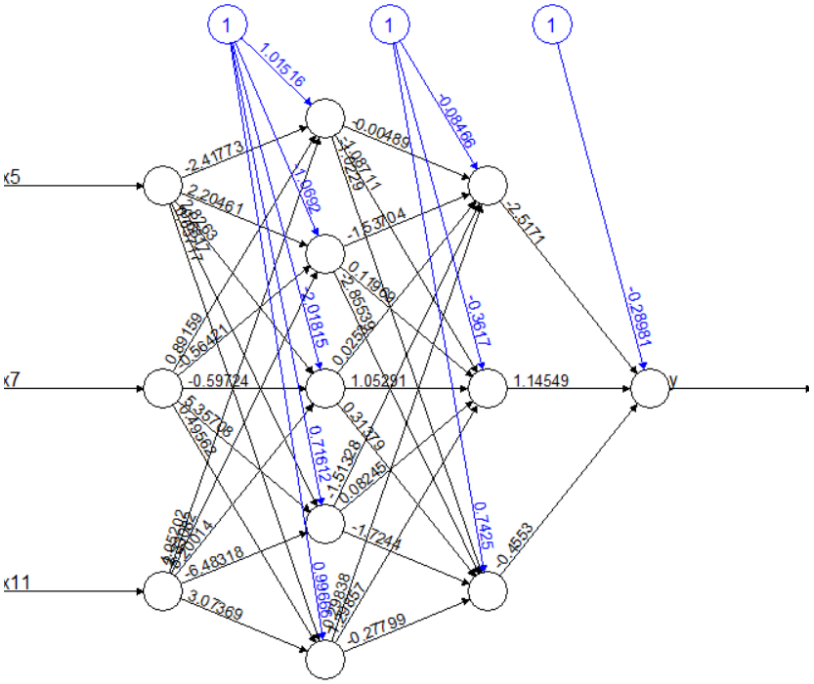
f	formula	y ~ x5 + x7 + x11
[[1]]	symbol	`~`
[[2]]	symbol	`y`
[[3]]	language	x5 + x7 + x11

Применяем метод нейросетей:

nn <- neuralnet(f,data=train_,hidden=c(5,3),linear.output=T), аргумент hidden принимает вектор с числом нейронов для каждого скрытого слоя (в примере их 3), в то время как аргумент linear.output используется для указания, хотим ли мы сделать регрессию (linear.output = TRUE) или классификацию (linear.output = FALSE)

Визуализируем нейросеть:

plot(nn)



Error: 0.587354 Steps: 140

Посмотрим результаты:

summary(nn)

```
                  Length Class      Mode
call                   5 -none-     call
response              59 -none-     numeric
covariate            177 -none-     numeric
model.list             2 -none-     list
err.fct                1 -none-     function
act.fct                1 -none-     function
linear.output          1 -none-     logical
data                   4 data.frame list
exclude                0 -none-     NULL
net.result             1 -none-     list
weights                1 -none-     list
generalized.weights    1 -none-     list
startweights           1 -none-     list
result.matrix         45 -none-     numeric
```

Нейросеть выдает нормализованный прогноз, поэтому нужно масштабировать его, чтобы провести значимое сравнение моделей регрессии нейросети и обобщенной модели.
Вычисляем теоретические значения с помощью нейросети на тестовой выборке:

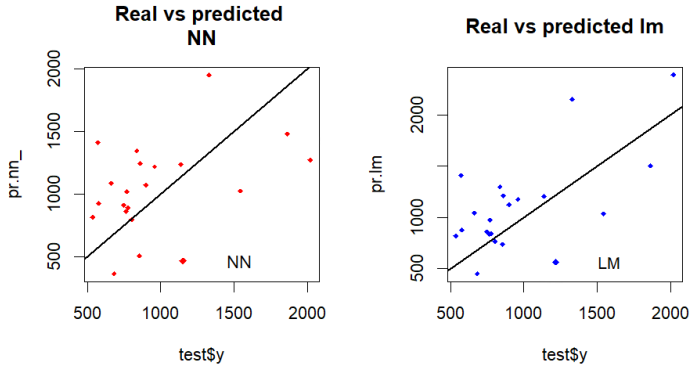
```
pr.nn=predict(nn,test_[,1:3])
```

Осуществляем маштабирование их:

```
pr.nn_ <- pr.nn*(max(data$y)-min(data$y))+min(data$y)
test.r <- (test_$y)*(max(data$y)-min(data$y))+min(data$y)
MSE.nn <- sum((test.r - pr.nn_)^2)/nrow(test_)
print(paste(MSE.lm,MSE.nn))
[1] "138757.244460412 164950.849346838"
```

Вывод: ошибка нейросети больше, то есть лучше использовать обобщенную модель.
Сравним графически по тестовой выборке:

```
plot(test$y,pr.nn_,col='red',main='Real vs predicted
NN',pch=18,cex=0.7)
abline(0,1,lwd=2)
legend('bottomright',legend='NN',pch=18,col='red', bty='n')
plot(test$y,pr.lm,col='blue',main='Real vs predicted lm',pch=18,
    cex=0.7)
abline(0,1,lwd=2)
legend('bottomright',legend='LM',pch=18,col='blue', bty='n', cex=.95)
```



Напишем уравнение регрессии нейросети: $У=-0,28-2,52X^{н}_{5}+1,15X^{н}_{7}-0,46X^{н}_{11}$